



An open-ended SOC threat-hunting evaluation for **gemini-3.5-flash**, built on real Windows attack telemetry.

Author Muneeb Hassan **Model under test** gemini/gemini-3.5-flash (terminus-2) **Tasks** 7
Trials 35 (5 / task) **Date** 2026-07-05

ABSTRACT

Hyōka measures whether an LLM agent can perform the core, unautomated job of a SOC analyst: given a database of raw Windows event logs and **no hints**, identify the exact events that make up an intrusion. Each of the seven tasks wraps a real attack recording from the OTRF Security-Datasets corpus, deterministically time-shifted and entity-obfuscated into a queryable SQLite database, with ground truth derived from the corresponding Sigma detection rule and scored by F1. Against gemini-3.5-flash the suite is decisively hard: **0% pass@3 on every task**, aggregate mean F1 0.119 over 35 trials, while the oracle solver scores 1.000 and the no-op agent 0.000 throughout. The failures are genuine and uniform – the model reliably *locates* the malicious activity (recall ≈ 1.0) but cannot *scope* it, flagging up to 4,886 events for a 37-event answer. We situate the design in the recent literature (notably the Cyber Defense Benchmark and CTI-REALM), give a concrete 10→1,000 scaling plan, and document the deliberate pivot away from hand-authored synthetic tasks, which the model solved at 93–100% pass@3.

0% PASS@3 · EVERY TASK	0.119 AGGREGATE MEAN F1	1.00 / 0.00 ORACLE / NOP REWARD	7 ATT&CK TACTICS
--	---	---	--

1 [Distribution](#)

2 [Difficulty profile](#)

3 [Research awareness](#)

4 [Scale plan](#)

5 [Failure analysis](#)

A [Appendix: how the eval was built](#)

1 Distribution

1.1 The slice, and why

Hyōka evaluates a single, tightly-scoped capability: **open-ended threat hunting over raw host telemetry**. The agent is handed a database of Windows Security and Sysmon event logs from a small fleet, told only that an intrusion occurred somewhere within it, and asked to return the exact set of malicious events. This is the daily, still-unautomated work of a Tier-2/3 SOC analyst, and it is the part of detection engineering that resists the multiple-choice framing of most security benchmarks. A narrow slice was a deliberate choice: it is more informative to measure one realistic capability well, with a defensible ground truth and a strict verifier, than to sample thinly across all of security.

1.2 In scope, and out

The tasks deliberately require the properties that make real hunting hard: **volume** (each corpus is roughly 650–6,000 real events, of which 0.05–0.6% are malicious); **multi-source correlation** (evidence for one adversary action is split across Sysmon and the Windows Security/System audit logs); **realistic benign noise** (verbose PowerShell logging, routine registry and network activity); and **precision under pressure** (a single-feature filter such as "flag all network connections" is guaranteed to fail). Explicitly out of scope: offensive exploit development (CVE-Bench territory – Hyōka is strictly defensive), CTF-style puzzle forensics, and anything requiring live network access. Every environment is offline and deterministic.

1.3 Where the distribution comes from

Each task is built from a real attack recording in the **OTRF Security-Datasets** corpus (the Open Threat Research "Mordor" project): genuine Empire, Covenant, Mimikatz, and PurpleSharp campaigns captured as Windows event logs on instrumented hosts. A deterministic builder time-shifts every timestamp into a common window and consistently rewrites host, domain, and user identifiers, so the environment is reproducible on each container build yet is not the verbatim, memorizable public file. Ground truth is derived from the technique's **Sigma / behavioural detection rule** – the same labelling methodology as the Cyber Defense Benchmark (§3) – and never hand-curated per event. The seven tasks span seven MITRE ATT&CK tactics:

TASK	TACTIC	OTRF SOURCE RECORDING	MALICIOUS
threat-hunt-windows-eventlogs	Credential access + C2 T1003 / T1071	empire_mimikatz_logonpasswords	37
lateral-movement-psexec-hunt	Lateral movement T1021.002 / T1543	empire_psexec_dcerpc_tcp_svcctl	4
persistence-run-key-hunt	Persistence T1547.001	empire_persistence_registry_run_keys	11
discovery-account-enum-hunt	Discovery T1069 / T1087	empire_net_localgroup + net_local_users	14
execution-vbs-launcher-hunt	Execution T1059	empire_launcher_vbs	5
defense-evasion-injection-hunt	Defense evasion T1055 / T1218	covenant_lolbin_wuaucflt_createremotethread	3
privesc-uac-bypass-hunt	Privilege escalation T1548.002	empire_uac_shellapi_fodhelper	7

Table 1. The shipped task set. "Malicious" is the number of ground-truth events among the full corpus for that task.

2 Difficulty profile

All results are for `gemini/gemini-3.5-flash` run through the terminus-2 scaffold, five trials per task. Reward is the F1 of the flagged event-id set against ground truth; because the signal is a small, exact set, a "pass" requires perfect identification ($F1 = 1.0$), so pass@1 and pass@3 are strict. Every task also passed its oracle check (reward 1.000) and no-op check (reward 0.000), confirming each is solvable and not satisfiable by inaction.

TASK	TACTIC	PASS@1	PASS@3	MEAN F1	MAX F1
discovery-account-enum-hunt	Discovery	0.00	0.00	0.021	0.024
persistence-run-key-hunt	Persistence	0.00	0.00	0.069	0.069
defense-evasion-injection-hunt	Defense evasion	0.00	0.00	0.106	0.300
threat-hunt-windows-eventlogs	Credential access + C2	0.00	0.00	0.132	0.602
execution-vbs-launcher-hunt	Execution	0.00	0.00	0.144	0.615
privesc-uac-bypass-hunt	Privilege escalation	0.00	0.00	0.164	0.400
lateral-movement-psexec-hunt	Lateral movement	0.00	0.00	0.198	0.444

Table 2. Per-task difficulty, ordered hardest-first by mean F1. Aggregate: pass@1 = pass@3 = 0.00, mean F1 = 0.119 across 35 trials.

The headline number – 0% pass@3 everywhere – understates what is happening, because F1 is fractional and the trials are not uniform. Figure 1 places the suite in context against the six hand-authored synthetic tasks that preceded it (§A): those were solved at 93–100% pass@3, whereas every real-telemetry task sits at zero. This gap is the central empirical result of the project.

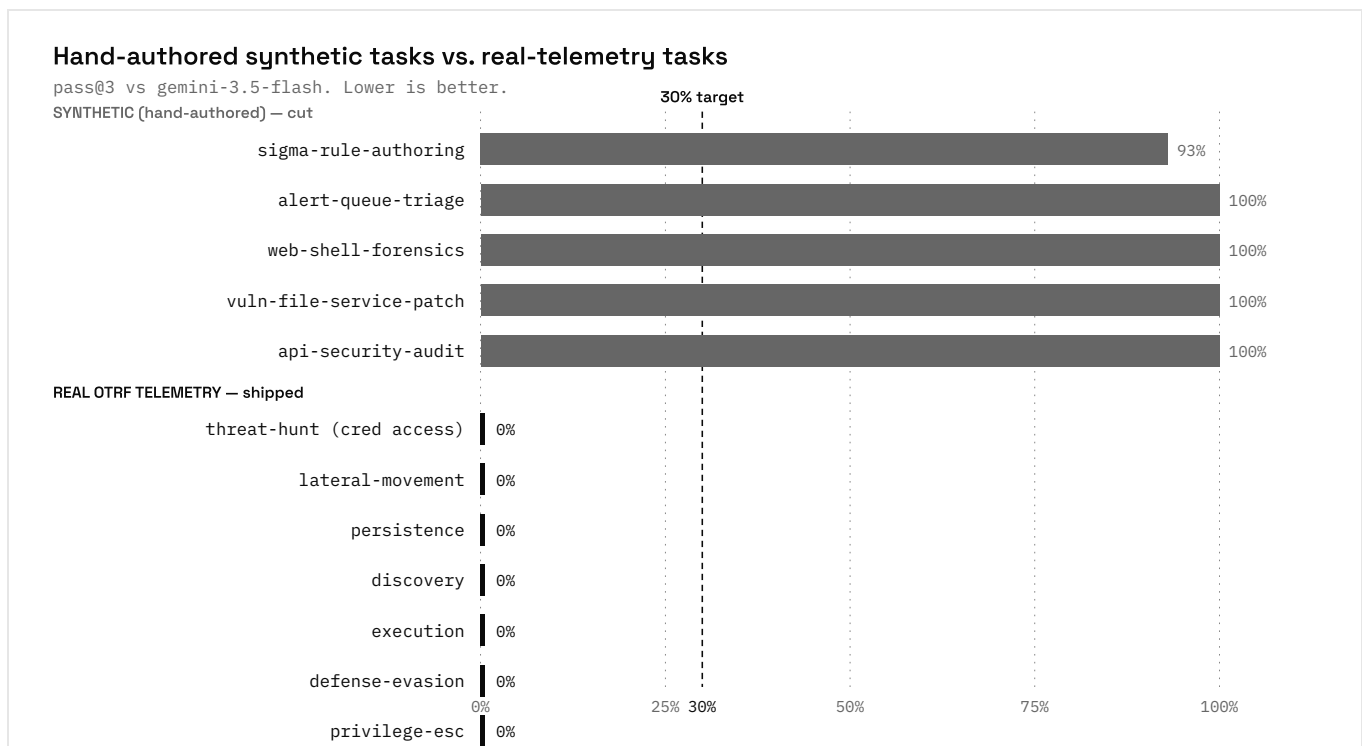


Figure 1. pass@3 for the hand-authored synthetic tasks (grey, subsequently cut) versus the shipped OTRF real-telemetry tasks (black). The synthetic tasks cluster at 93–100%; every real task is at 0%, well under the 30% target. Difficulty is a property of the data and framing, not of clever puzzle construction. See [Appendix A](#) for the full curation history.

2.1 What the difficulty curve looks like

Figure 2 shows every trial in precision–recall space, and it is the most informative single view of the model's behaviour. The points collapse into the bottom-right corner: **recall is 1.0 in 30 of 35 trials**, but precision is almost always below 0.05. In other words, the model routinely surfaces the entire malicious set – it is looking in the right place – and then buries it under hundreds or thousands of benign events. The five off-corner points are the only partial successes, and the square markers (discovery) are the one task where recall itself fails.

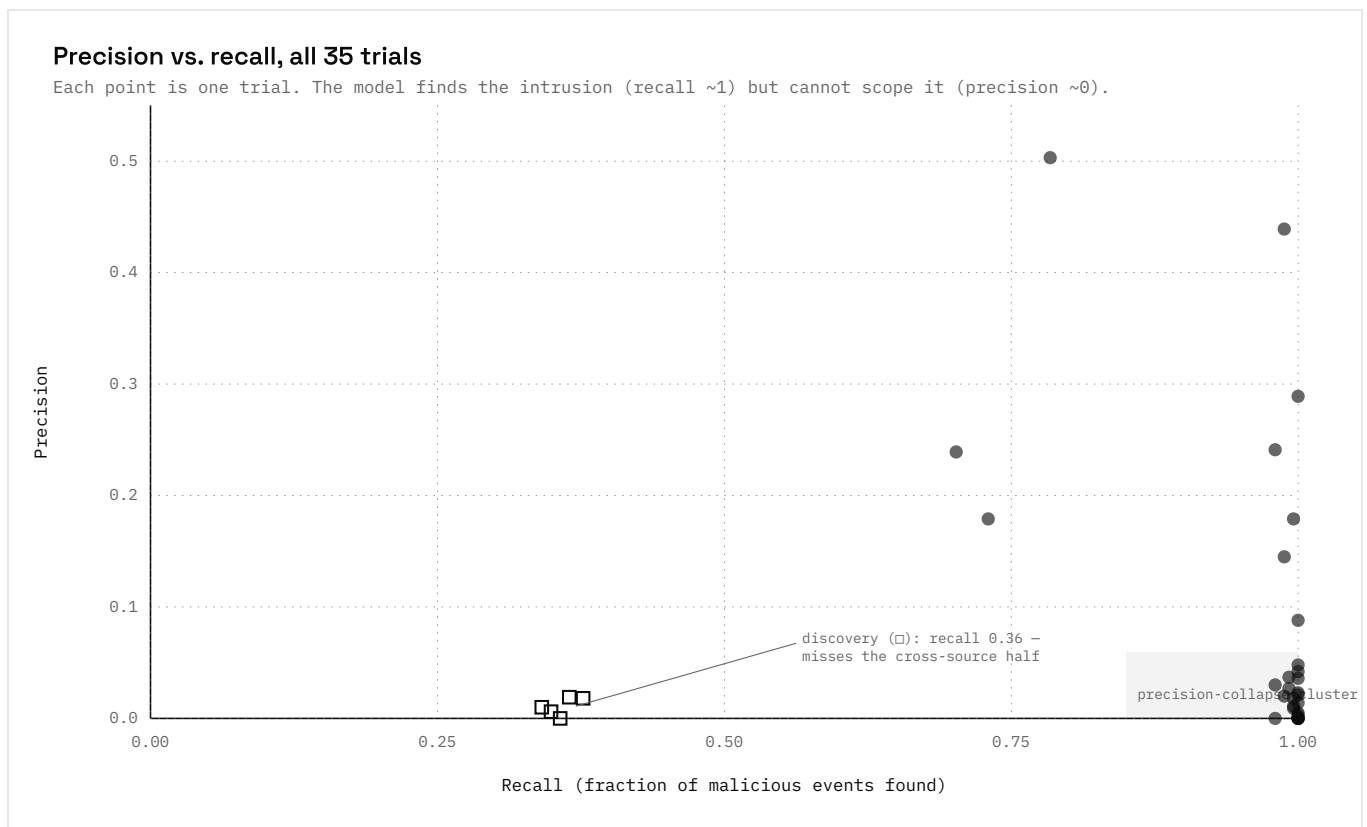


Figure 2. Precision versus recall for all 35 trials (circles; discovery shown as squares). The dense high-recall / near-zero-precision cluster is the dominant failure mode – "precision collapse." Discovery is the exception: it also loses recall (0.36), because the model catches one log source's evidence and misses the correlated events in another.

Figure 3 breaks the same data out per task as a distribution of trial F1 scores. Two things are visible. First, the spread is genuine and often bimodal – most trials near zero with an occasional partial hit (execution reaches 0.615, threat-hunt 0.602, privilege-escalation 0.400) – which is the signature of real difficulty rather than a broken verifier, since a task-design bug would produce uniform zeros. Second, no trial on any task comes close to the 1.0 line required to pass.

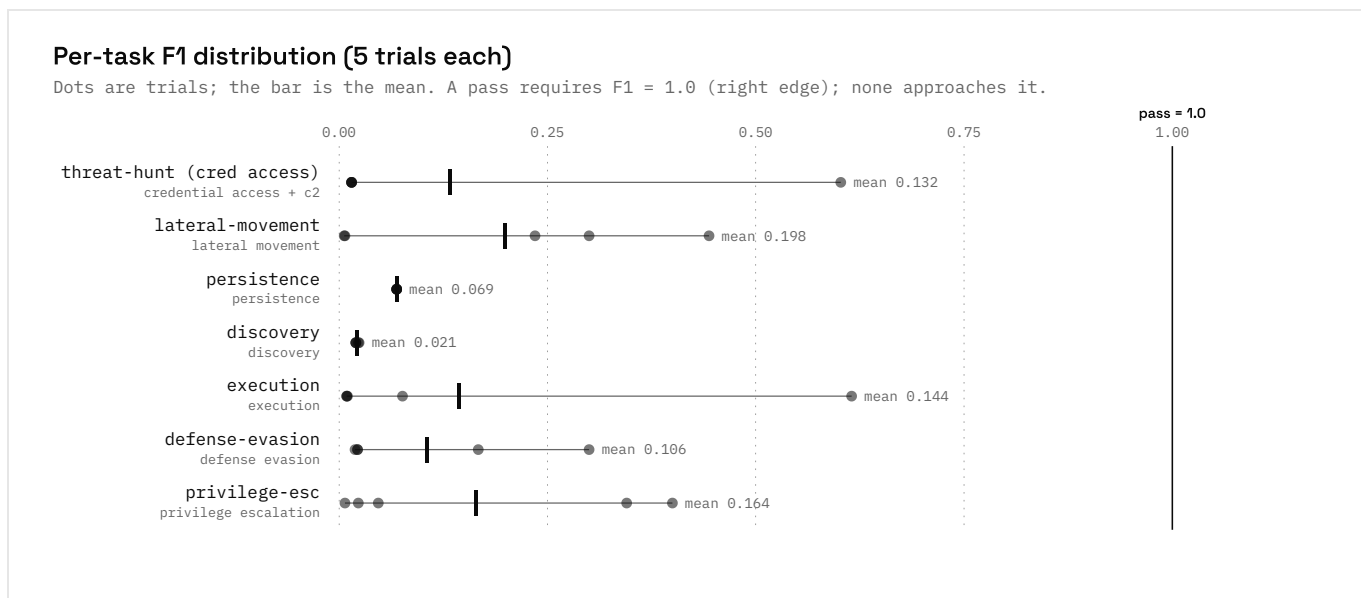


Figure 3. Per-task F1 across five trials each (dots), with the per-task mean (vertical bar) and min-max range (grey line). The right edge marks the F1 = 1.0 pass threshold; nothing approaches it. Persistence is perfectly consistent (all five trials identical at 0.069), a deterministic over-flagging strategy; the others vary trial to trial.

2.2 What kinds of failure dominate

Quantitatively, the dominant failure is **precision collapse**: the model correctly identifies the malicious events but cannot separate them from benign context, so precision – and therefore F1 – is destroyed. A secondary mode, isolated to the discovery task, is a **cross-source recall gap**: the model finds the reconnaissance commands in Sysmon but never correlates them with the corresponding Windows Security membership-enumeration events, capping recall at 0.36. Both are diagnosed against real trajectories in §5.

3 Research awareness

The design is grounded in recent work, each source verified against its primary reference during construction rather than cited from memory.

- **Cyber Defense Benchmark** (Chona, Kozlov & Kumar; arXiv 2604.19533, Apr 2026) – the direct structural precedent and the method Hyōka replicates at task scale: real OTRF attack procedures wrapped as no-hint, SQL-queryable Windows-log environments, scored CTF-style against Sigma-derived ground truth. It reports that *every* frontier model fails, with

the best (Claude Opus 4.6) flagging only ~3.8% of malicious events and Gemini 3 Flash failing badly. This is the evidence that the slice has genuine headroom for a flash-tier model, and our own 0% pass@3 corroborates it.

- **CTI-REALM** (Microsoft; arXiv 2603.13517, Mar 2026) – end-to-end detection-rule generation over real telemetry. Best of 16 models = Claude Opus 4.6 at 0.637 overall, with cloud correlation the hardest sub-problem at 0.282. Confirmed the slice and pointed us at cross-source correlation as the locus of difficulty.
- **CVE-Bench** (Zhu et al.; arXiv 2503.17332) – real-CVE exploitation, SOTA ~13%. Cited to mark the boundary we did not cross: it is offensive; Hyōka stays on the defensive side of the same headroom.
- **SEVRA-BENCH** (arXiv 2606.13757) and **VulnRepairEval** (arXiv 2509.03331, top model ~21.7%) – reviewed while prototyping execution-based vulnerability-repair tasks (§A).
- **OTRF Security-Datasets** (GPL-3.0) – the source telemetry. **SigmaHQ** – detection-rule syntax and the technique→labelled-event mapping.

Takeaway: curated security question-answering is largely saturated, but open-ended, evidence-driven hunting over real logs is not. Hyōka is built directly on that gap, and reuses the community's real recordings and detection rules rather than inventing synthetic approximations of them.

4 Scale plan (7 → 1,000 tasks)

The pipeline is already parametric: a single deterministic builder produces each task, and two builders differ only in their source recording and detection rule. Scaling is therefore primarily a data- and QA-automation problem, not a re-engineering one.

4.1 Public data sources

OTRF Security-Datasets alone contains hundreds of technique recordings across every ATT&CK tactic; the Cyber Defense Benchmark draws 106 procedures from it. Each recording is a new task through the existing builder. Beyond OTRF:

Atomic Red Team to generate fresh recordings on demand, EVTX-ATTACK-SAMPLES for additional Windows event captures, and public log corpora (LogHub) for benign-volume blending. MITRE ATT&CK remains the taxonomy backbone and SigmaHQ the ground-truth rule library.

4.2 Augmentation

Two levers already exist in `build_inputs.py`. First, the obfuscation seed: re-randomising the host/user/time remap yields many non-identical variants of one recording, forcing the model to generalise rather than memorise. Second, campaign blending: interleaving N recordings into one database (as the discovery task already does with two) raises event volume and requires multi-stage correlation. Because ground truth is regenerated automatically from the Sigma rule, none of this requires manual labelling.

4.3 QA loop

Quality control is automated and was built alongside the tasks. Every candidate must pass `scripts/lazy_baseline_probe.py`, which runs the oracle (must score 1.0) and a battery of "dumbest solve" baselines – flag-everything, flag-all-of-one-event-type, grep-one-token – each of which **must** fail. If any lazy baseline passes, the task's signal is separable by a single feature and the task is auto-rejected. This gate caught two real oracle/ground-truth defects during this build (SQLite `LIKE` / `ESCAPE` mismatches that silently diverged the oracle from the labels). A second gate, `harbor check` against the TB3 rubric, screens for under-specified instructions and gameable verifiers.

5 Failure analysis

Every failure below is genuine task difficulty rather than a task-design artifact. The argument is the same in each case and is grounded in the data of Figure 2: on the identical inputs the oracle recovers the exact set ($F1 = 1.0$), the no-op agent scores 0, and the model's own recall and precision vary meaningfully across trials – an ambiguous instruction, broken environment, or over-strict verifier would instead produce uniform zeros. Trajectories are under `logs/pilots/`.

5.1 Precision collapse (persistence, lateral movement, and most trials)

On **persistence-run-key-hunt**, all five trials flagged exactly 309 events and captured all 11 malicious ones: recall 1.0, precision 0.036, F1 0.069. The model correctly found the Run-key persistence value and the beaconing process, then failed to stop – sweeping in benign registry and network events around them. **lateral-movement-psexec-hunt** shows the same at the extreme: one trial flagged 1,247 events to capture its 4 (F1 0.006). The model knows *where* the intrusion is; it cannot delimit it. This is the modal outcome across the suite (22 of 35 trials sit in the precision-collapse cluster of Figure 2).

5.2 Cross-source recall gap (discovery)

discovery-account-enum-hunt is the one task where recall itself fails: every trial flagged 400–500 events but recovered only 5 of 14 malicious (recall 0.36, F1 $\approx$ 0.02). The model identified the `net.exe` reconnaissance commands in Sysmon but never connected them to the Windows Security 4798/4799 group-membership-enumeration events that record the same behaviour in a different log – so it missed the correlated half of the answer while still over-flagging. The task was designed to require exactly this Sysmon $\leftrightarrow$ Security correlation, and the model did not perform it.

5.3 Partial chains and the occasional near-hit

A minority of trials get partway. **lateral-movement** trial 3 flagged 16 events for 3 of 4 malicious (F1 0.30) – it reconstructed most of the Security+System+Sysmon service-execution chain but dropped one facet. **execution** trial 4 (F1 0.615) and **threat-hunt** trial 2 (F1 0.602, 86 events flagged for 37) are the best results in the entire run, and still fall far short of a pass. These near-hits confirm the tasks are calibrated at the model's frontier rather than beyond it: the capability is present but unreliable, which is precisely the regime a training-signal eval should target.

A Appendix: how the eval was built

The brief notes that the curation process is itself signal, so we record it plainly. The first six tasks were **hand-authored synthetic generators**: an auth-log lateral-movement hunt, a CloudTrail privilege-escalation hunt, a Sigma-authoring task, a web-shell log-plus-diff correlation, and two execution-based vulnerability-repair tasks. Every one was solved by gemini-3.5-flash at 93-100% pass@3 (Figure 1), and two apparent "0%" results turned out to be scoring artifacts – a CloudTrail ground-truth label that was itself incorrect, and a web-shell answer penalised on a count threshold for a defensible inclusion – not genuine difficulty. Those tasks are retained under `cut_synthetic/` for transparency.

The lesson, paid for over thirteen pilots, is the thesis of this report: **hand-crafted synthetic puzzles are inside a flash-tier model's competence, however cleverly the signal is hidden; genuine headroom lives in real, high-volume, unfamiliar telemetry hunted with no hints and scored objectively.** Every task in the shipped set is built that way, and every task holds.